

STARWU-LITE 开发框架

软件设计说明书

1 引言

1.1 软件简介

【STARWU-LITE 开发框架 V1.0】是一款基于 Java 语言自研的轻量级开发框架。

本框架主要面向 Java 后端开发场景，使一个相对简陋的项目，通过集成该框架，成为一个非常成熟的项目。

本框架运行依赖 JDK 环境，不包含可视化 GUI 界面，以 Jar 包形式给业务项目引入使用。

1.2 开发目的

自从 SpringBoot 框架问世以后，受到了很多 java 开发人员的欢迎，其中一个很重要的原因就是它简化了很多的配置，使我们可以快速的搭建一个项目。我们只要用 maven 把 SpringBoot 的 jar 包引入进来，然后在写个启动类，在写一个 Controller，就可以启动项目，并且对外提供服务了。

但是问题是，这样的项目还是一个很简陋的项目，还远远不够成熟，还有很多很多的工作需要我们去处理，比如：请求日志如何记录、响应参数如何封装、异常怎样统一处理、threadLocal 对象如何清除、数据库与 redis 如何配置使用、怎样避免 redis 的缓存击穿、缓存穿透问题、怎么处理 redis 与数据库的数据一致性的问题，如果用到分布式了，分布式事务、分布式 session 怎么解决、如何监控慢 sql，等等等等，这些问题都需要我们去处理。

这些问题看起来都不是特别难以处理，但是如果每个问题都需要我们去自己处理的话，也是一件很麻烦的事情。

而本框架，就是为了去解决这些问题，只要在刚刚搭建的 SpringBoot 项目中，引入本框架的 jar 包，就可以将一个很简陋的项目，升级到一个足够成熟的项目，包括且不限于上面提到的问题，都在框架中帮助我们处理了，所以，我们就可以利用本框架，快速的搭建起一个很成熟的项目。

本软件著作权保护范围为本项目自研编写的全部源代码，框架运行所依赖的 JDK、Maven 以及其他第三方开源组件，不属于本软件著作权保护范围。

1.3 应用领域

本软件适用于 Java 后端服务开发、小型服务项目等场景。

1.4 运行环境

硬件环境

CPU: Intel/AMD 双核及以上;

内存: 4GB 及以上;

硬盘: 100MB 以上可用存储空间。

软件环境

开发语言: Java

JDK 版本: JDK8

构建工具: Manev 3.3.3

运行操作系统: Windows10/11、Linux (CentOS7、Ubuntu)

2 软件总体架构

2.1 整体架构

本框架采用模块化设计，整体分为以下几个模块：

1. **START 模块：**框架的统一入口。
2. **WEB 模块：**主要负责对 web 请求做一些封装，比如记录日志、统一异常处理、响应参数的封装等等。
3. **DAO 模块：**基于 mybatisPlus，封装了 mybatisPlus 与 redis 的结合使用得过程，以及处理 redis 与数据库数据一致性的问题，和 redis 缓存击穿、缓存穿透等问题。和监控慢 sql，以及记录数据库变更日志等功能。
4. **CANAL 模块：**通过集成阿里的 canal 中间件，监听数据库的所有变更记录，并且推送给所有的订阅者。订阅者可以依据此记录来解决一系列问题，比如数据一致性的问题、记录数据库变更记录等等等等。
5. **DISTRIBUTED 模块：**负责解决分布式带来的一系列问题，目前解决了分布式 session 和分布式事务的问题。
6. **DESIGN 模块：**负责对设计模式做一些封装，目前封装了工厂模式、责任链模式、观察者模式。
7. **MANAGE 模块：**负责对一对象资源做统一管理，比如 ThreadLocal 对象的统一管理，Spring 的异步资源的统一管理等等。
8. **METADATA 模块：**定义各个模块都会用到的一些元数据信息，比如：注解、枚举、实体类、配置信息等等等等。

9. **UTIL 模块：**定义一些常用的工具类。

2.2 架构图

该框架的架构图如下：

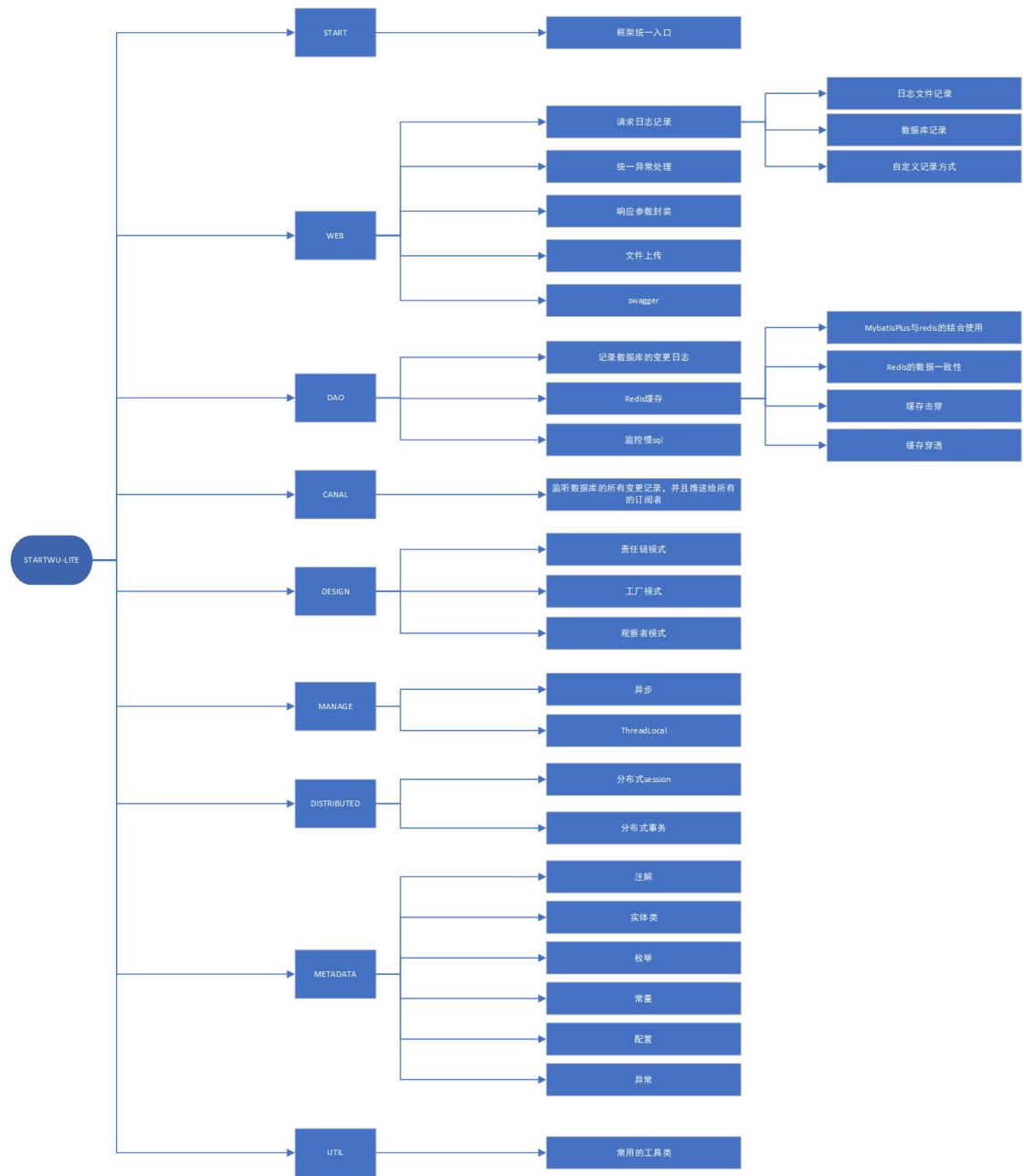


图 2-1 软甲架构图 1

3 详细设计说明

3.1 整体设计

3.1.1 框架引入

本框架采用 maven 模块化开发，根据对外提供的功能的不同，来划分不同的 module，具体的划分规则，已经在上文中描述。

其中 START 模块是该框架的入口，在该模块中，没有任何的代码，它只是把其他所有的模块都依赖进来，如下图 3-1 所示。

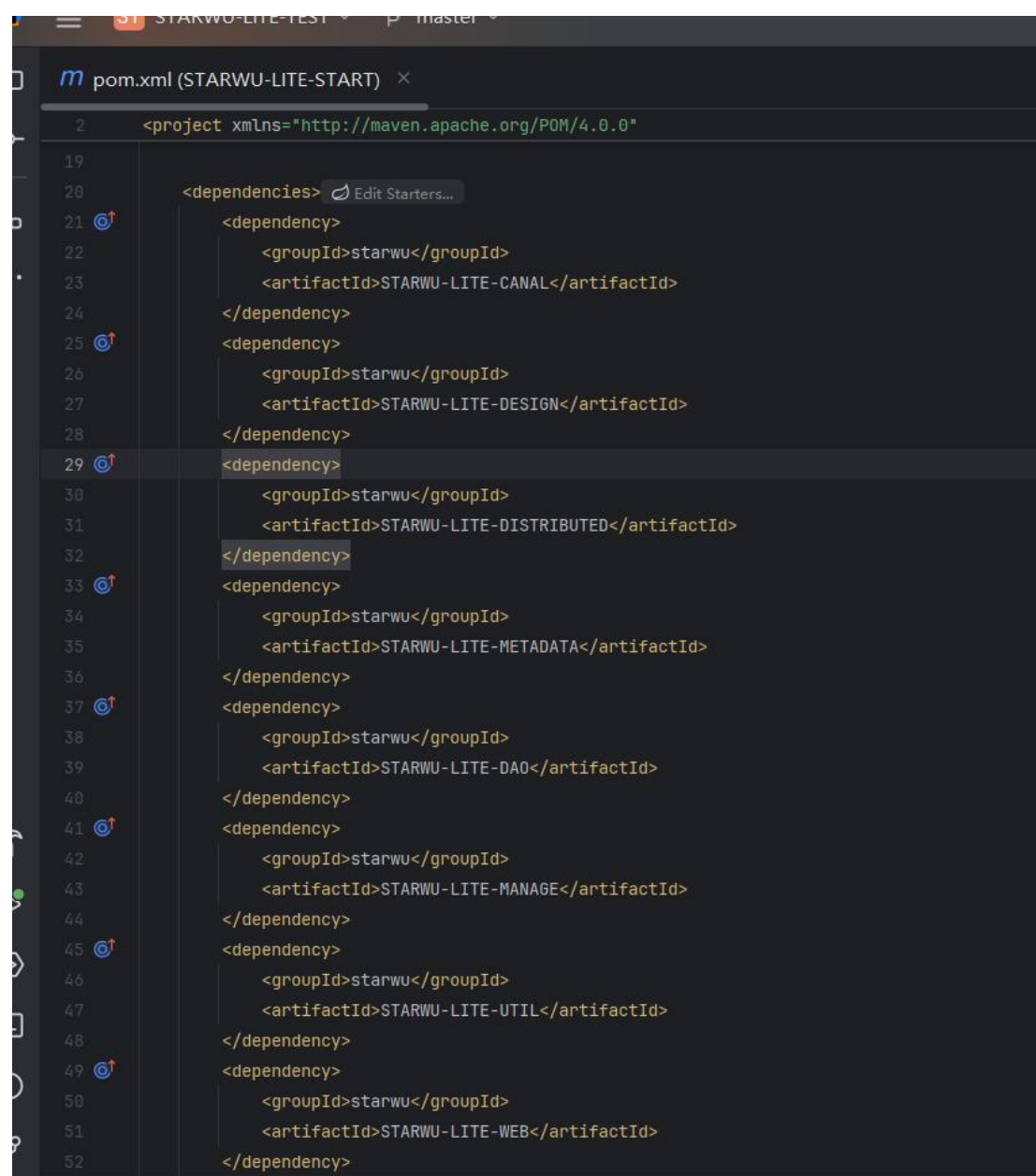


图 3-1 STARWU-LITE-START 模块依赖

在使用该框架时，直接引入 START 模块即可，如下图 3-2 所示。

```
<dependencies> Edit Starters...
  <dependency>
    <groupId>starwu</groupId>
    <artifactId>STARWU-LITE-START</artifactId>
    <version>1.0.0</version>
  </dependency>
</dependencies>
```

图 3-2 引入 STARWU-LITE-START 模块

这样引入 START 模块以后，就可以使用该框架提供的全部的功能了。

但是如果在使用的時候，我只是想使用框架提供的其中一部分功能，比如我只想使用 WEB 模块的记录请求日志的功能，其他的功能我都不用，这样其实就没有必要把所有的模块都引入进来，就只需要把 WEB 模块引入引来就可以，如下图 3-3 所示：

```
<dependencies> Edit Starters...
  <dependency>
    <groupId>starwu</groupId>
    <artifactId>STARWU-LITE-WEB</artifactId>
    <version>1.0.0</version>
  </dependency>
</dependencies>
```

图 3-3 引入 STARWU-LITE-WEB 模块

3.1.2 bean 扫描

在本框架中的除 START 模块的其他所有模块，在 resources 中都会有一个 org.springframework.boot.autoconfigure.AutoConfiguration.imports 文件，它是 SpringBoot 提供的功能，用来指定扫描框架中定义的 bean 的。在该框架中，每个 imports 文件，都会指向一个 bean，然后在该 bean 中，用 @Import 注解指定要扫描的该模块中的哪些 bean，以 WEB 模块举例，如下图 3-4 和 3-5 所示：

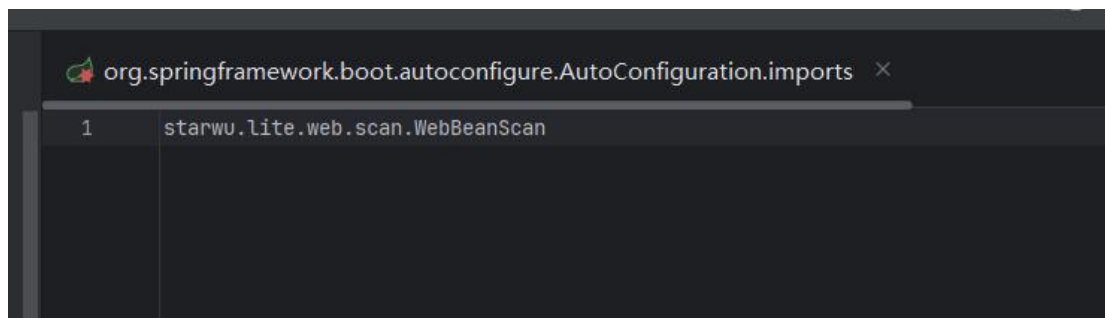


图 3-4 imports 文件

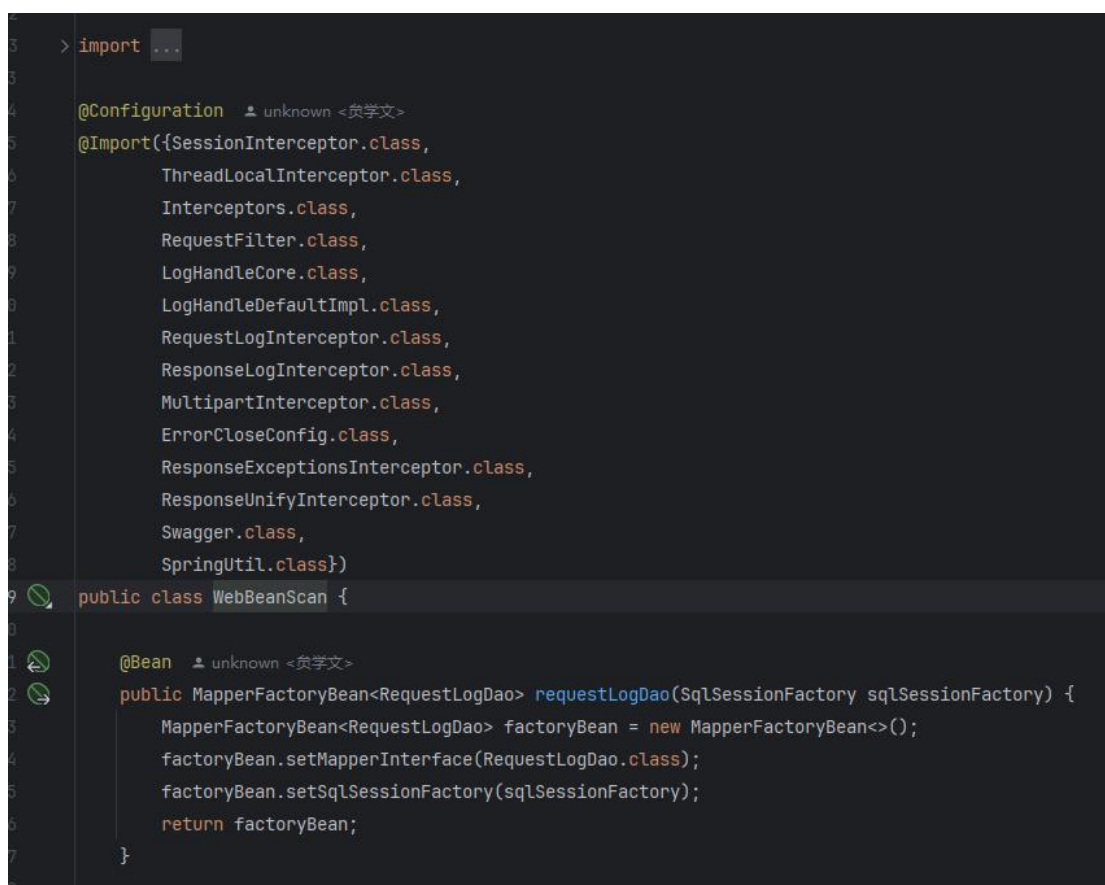


图 3-5 配置要扫描的 bean

这样，在使用该框架时，无论是直接引入 START 模块，还是引入了其他的模块，都无需考虑 bean 扫描的问题，bean 的扫描，都在框架中处理好了。

3.1.3 外部 jar 包依赖

本框架中，也依赖了一些其他的 jar 包，所有依赖的 jar 包和依赖的 jar 包版本，都在父 module 中的 pom.xml 中的<properties>中，做统一管理，如下图 3-6 所示：

```
<properties>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
  <spring-boot.version>2.7.14</spring-boot.version>
  <fastjson2.version>2.0.62</fastjson2.version>
  <canal.version>1.1.7</canal.version>
  <mybatis-plus.version>3.5.17</mybatis-plus.version>
  <redisson.version>3.17.7</redisson.version>
  <UserAgentUtils.version>1.21</UserAgentUtils.version>
  <springfox-swagger2.version>2.9.2</springfox-swagger2.version>
  <swagger-annotations.version>1.5.20</swagger-annotations.version>
  <springfox-swagger-ui.version>2.9.2</springfox-swagger-ui.version>
  <knife4j-spring-ui.version>2.0.9</knife4j-spring-ui.version>
  <swagger-bootstrap-ui.version>1.9.4</swagger-bootstrap-ui.version>
  <starwu.lite.version>1.0.0</starwu.lite.version>
</properties>
```

图 3-6 jar 包版本管理

如果在使用该框架的过程中，觉得框架中依赖的某个 jar 包的版本太低或者太高，想去对版本做升级或者降级，可以在引入该框架时，用<exclusions>标签移除该 jar 包，然后在引入你想要的 jar 包即可。

3.2 WEB 模块

3.2.1 记录请求日志

➤ 记录请求日志的目的：

1. 某个接口出现问题时，可以根据请求日志快速定位问题。
2. 可以监控接口响应的成功率以及响应时长，从而制定出系统的优化方案。
3. 可以方便的统计用户访问某些接口的频率，从而制定出客户画像。

3.2.2 响应参数封装

3.2.3 统一异常处理

3.2.4 文件上传

3.2.5 swagger

3.3 DAO 模块

3.3.1 数据库与 redis 的结合使用

3.3.2 redis 的缓存击穿

3.3.3 redis 的缓存穿透

3.3.4 redis 与数据库的数据一致性

3.3.5 记录数据库的变更日志

3.3.6 监控慢 sql

3.3.7 mybatisPlus 的分页

3.4 CANAL 模块

3.4.1 监听数据库的变更

3.4.2 将数据库的变更推送给订阅者

3.5 DISTRIBUTED 模块

3.5.1 分布式 session

3.5.2 分布式事务

3.6 DESIGN 模块

3.6.1 工厂模式

3.6.2 责任链模式

3.6.3 观察者模式

3.7 MANAGE 模块

3.7.1 ThreadLocal 功能增强

3.7.2 Spring 异步功能增强

3.8 UTIL 模块

4 核心类与接口说明

模块	类 / 接口名称	所在包路径	核心功能简述
WEB 模块	RequestLogInterceptor	starwu.lite.web.log.interceptor	在日志文件中记录请求日志
WEB 模块	ResponseLogInterceptor	starwu.lite.web.log.interceptor	在日志文件中记录响应日志
WEB 模块	LogHandleDefaultImpl	starwu.lite.web.log.handler	在数据库中记录请求与响应日志
WEB 模块	LogHandleApi	starwu.lite.web.log.handler	通过实现此接口，来实现自定义记录请求日志
WEB 模块	ResponseExceptionHandler	starwu.lite.web.response	统一异常处理
WEB 模块	ResponseUnifyInterceptor	starwu.lite.web.response	响应参数的统一封装
WEB 模块	MultipartInterceptor	starwu.lite.web.multipart	实现文件上传
WEB 模块	Swagger	starwu.lite.web.swagger	集成 swagger
ORM 模块	BaseDaoExt	starwu.lite.orm.dao	数据库与 redis 的结合使用
ORM 模块	CanalRedisDefaultHandleImpl	starwu.lite.orm.dataConsistency.impl	解决数据库与 redis 的数据一致性
ORM 模块	CanalDblogHandlerDefaultImpl	starwu.lite.orm.dblog.impl	记录数据库数据的变更日志
ORM 模块	DaoSqlInterceptor	starwu.lite.orm.interceptor	监控数据库的慢sql
CANAL 模块	TcpListener	starwu.lite.canal.listener	监听 canal 的数据库的变更信息
CANAL 模块	CanalSubject	starwu.lite.canal.notify	将监听到的数据库变更信息通知到订阅者
DISTRIBUTED 模块	SessionCore	starwu.lite.distributed.session.core	解决分布式 session 的问题
DISTRIBUTED 模块	TransactionCore	starwu.lite.distributed.transaction.core	解决分布式事务的问题
MANAGE 模块	AsyncThreadPool	starwu.lite.plus.async.threadPool	Spring 异步功能的增强
MANAGE 模块	SnowFlakePlus	starwu.lite.plus.snowflake	雪花算法的增强
MANAGE 模块	ThreadLocalPlus	starwu.lite.plus.threadLocal	ThreadLocal 的增强

5 小结

本 STARWU-LITE 开发框架 V1.0 为基于 Java 的自研开发框架，实现了将一个简陋的项目瞬间变成一个成熟的项目的功能，以 Jar 组件库的形式提供给业务项目集成，简化后端开发流程。

本软件所有核心代码均为独立开发完成。

如果在使用遇到，请联系作者，作者姓名：负学文，联系方式/微信：18612230443。